

Lecture: Selection Sort Algorithm — From Zero to Advanced

1. Introduction

Selection Sort is one of the simplest **comparison-based sorting algorithms**.

It works by repeatedly finding the **minimum element** from the unsorted part of the list and placing it at the correct position in the sorted part.

Definition:

Selection Sort divides the array into two parts: **sorted** and **unsorted**. It repeatedly selects the smallest (or largest) element from the unsorted portion and swaps it with the first unsorted element.

Real-life analogy:

Imagine arranging books by height on a shelf. Each time, you **pick the shortest book** from the pile and place it at the beginning, repeating until all are sorted.

2. Key Characteristics

- **Type:** Comparison-based Sorting
- **Input Requirement:** Works on any list (unsorted or sorted)
- **Direction:** Selects the smallest element each time and places it at the beginning
- **Time Complexity:**
 - Best: $O(n^2)$
 - Average: $O(n^2)$
 - Worst: $O(n^2)$
- **Space Complexity:** $O(1)$ (in-place sorting)
- **Stability:** Not stable by default (can be modified to be stable)

3. Step-by-Step Working

1. Start with the first position.
2. Find the smallest element from the unsorted part of the list.
3. Swap it with the element at the first position.
4. Move the boundary of the sorted part one step forward.
5. Repeat until the entire list is sorted.

4. Pseudocode

SelectionSort(arr):

 n = length(arr)

 for i = 0 to n-2:

 min_index = i

 for j = i+1 to n-1:

 if arr[j] < arr[min_index]:

 min_index = j

 swap(arr[i], arr[min_index])

5. Complexity Analysis

- **Best Case:** $O(n^2)$ → even if already sorted, still scans whole list
- **Worst Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Space Complexity:** $O(1)$ (in-place)
- **Stability:** No (can be unstable due to swaps)

6. Handling Edge Cases

1. **Empty list** → nothing to sort
2. **Single element list** → already sorted
3. **All elements identical** → no effect, still $O(n^2)$ comparisons
4. **Partially sorted list** → still $O(n^2)$, no optimization benefit

7. Variations of Selection Sort

- **Stable Selection Sort** → by shifting elements instead of swapping.
- **Heap Sort** → can be thought of as an improved Selection Sort using heaps.
- **Bidirectional Selection Sort (Double Selection Sort):** Finds both min and max in each pass to reduce iterations.

8. When to Use Selection Sort

Best suited for:

- Small datasets.
- Situations where memory is extremely limited (only $O(1)$ extra space).
- When the number of swaps must be minimized (fewer swaps than Bubble Sort).

Avoid if:

- Large datasets (inefficient due to $O(n^2)$ complexity).
- Cases where stability is required.

9. Advanced Usage

- Sometimes used when **swap cost is high** but comparisons are cheap.
- Teaching purposes to explain **sorting fundamentals**.
- Base concept for more advanced algorithms (Heap Sort).

10. Real-World Applications

- **Education:** Teaching sorting basics.
- **Embedded systems:** Simple implementation with minimal memory use.
- **Small datasets:** Quick to implement where efficiency is not critical.
- **Hardware-level sorting:** Sometimes applied where swaps are expensive.

11. Summary Table

Feature	Selection Sort
Works on sorted?	Yes
Works on unsorted?	Yes
Time Complexity	Best: $O(n^2)$, Worst: $O(n^2)$, Avg: $O(n^2)$
Space Complexity	$O(1)$
Stable?	No (by default)

12. Practice Tasks

1. Implement basic Selection Sort in Python.

2. Modify Selection Sort to make it **stable**.
3. Implement **descending order Selection Sort**.
4. Compare Selection Sort with Bubble Sort and Insertion Sort on datasets of size 1000.
5. Implement **Bidirectional Selection Sort** (finding min & max in each pass).

13. Closing Thought

Selection Sort is simple but inefficient for large datasets.

Its value lies in its **clarity and simplicity**, making it excellent for **educational purposes**.

Although rarely used in practice, understanding Selection Sort builds a foundation for learning more efficient algorithms like **Heap Sort and Quick Sort**.